Bre-B: detección de fraude transaccional en tiempo real

Qué construimos, por qué así y qué aprendimos

PREGUNTA CENTRAL

Dado el fraude en pagos inmediatos, ¿qué pipeline de datos construiríamos y por qué exactamente así?

SA **Samuel Arango**
Ingesta y streaming

MS **Mateo Sanz Medina**
Almacenamiento

NC **Nathalia Valentina Cardoza Azuaje**
DataOps

SM **Sebastian Andres Medina Cabezas**
Consumo y gobernanza

Profesor Andrés Sacre Alzate · Medellín

En Bre-B el fraude se consuma en minutos

20 s

tarda en acreditarse una transferencia, 24/7

1.700 millones

transacciones en el primer año

10,4 millones

operaciones en el día récord: unas 120 por segundo

Entidad financiera

recibe la alerta en SQS y retiene los fondos

Analistas de fraude

confirman redes y reportan a la UIAF

Cumplimiento

reconstruye el linaje de una alerta (SFC)

> *Detección asíncrona con notificación: el sistema no participa en la autorización del pago; decide y avisa en paralelo.*

Cuatro preguntas de negocio; la decisión la toma el modelo

1

¿El ordenante envía a muchos destinatarios nuevos en poco tiempo?

→ Ventanas de 5 min por ordenante

2

¿Hay concentración hacia cuentas mula?

→ Ventanas de 10 min por receptora

3

¿La cuenta reenvía rápido lo que recibe?

→ Variables del modelo, con el estado de la cuenta

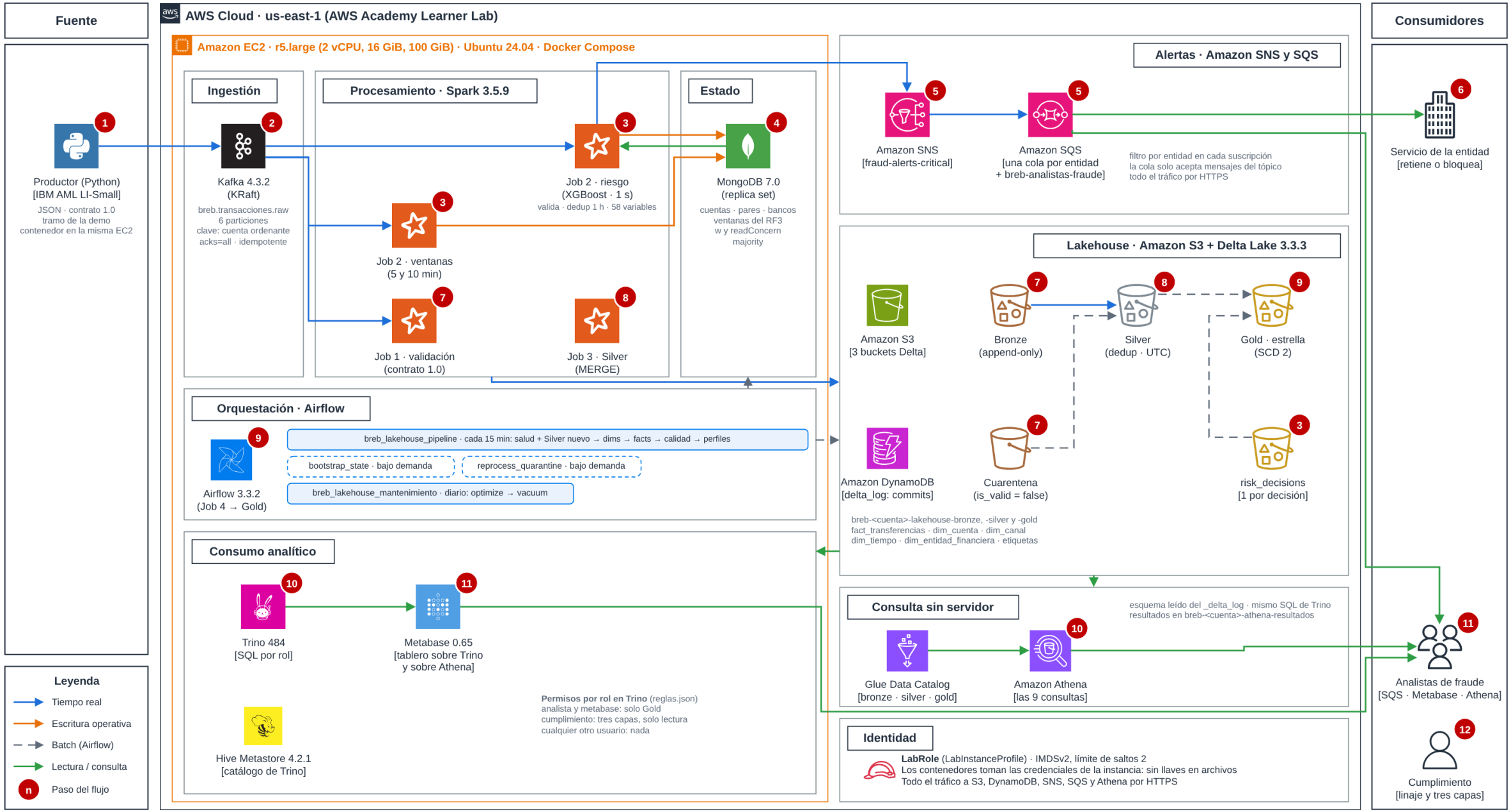
4

¿Qué tasa de alertas hay por entidad y por canal?

→ Gold, en el tablero de Metabase

Las preguntas no son reglas: la decisión la toma un modelo de IA (diapositiva 7)

Bre-B · Detección de fraude en tiempo real · Arquitectura desplegada en AWS (MVP en AWS Academy · us-east-1)



Del Entregable 1 al 2: qué se mantuvo, qué cambió y qué falló

Pieza	Diseño del E1	Desplegado en AWS	Por qué
Ingesta	API en ECS Fargate	El productor publica directo en Kafka	La API no tenía un consumidor real
Kafka, Spark y Airflow	Amazon MSK, EMR Serverless y MWAA	Kafka 4.3.2, Spark 3.5.9 y Airflow 3.3.2 en una EC2 r5.large	AWS Academy limita los servicios y las instancias (2 vCPU); pasarlos a gestionados exige cambios de código
Estado	MongoDB Atlas	MongoDB 7.0 (replica set) en la EC2	Mismo motor, sin cuenta externa
Consulta y permisos	Athena con Lake Formation	Athena y Glue Data Catalog; Trino para los permisos por rol	Lake Formation no está en el laboratorio
risk_decisions	Dentro del micro-batch	Fuera del micro-batch: un commit cada 10 s	Escribirlo dentro costaba de 6 a 19 s por micro-batch

Se mantuvo

Kafka · Spark Structured Streaming · MongoDB CP · Delta en S3 con commits en DynamoDB · estrella con SCD 2 · SNS y SQS · Athena y Glue · Airflow

Falló

La latencia de la decisión

RNF-02 · p99 < 3 s

Decisión: Kafka y Spark, con el modelo dentro del Job 2

Spark Structured Streaming y no Flink

Un solo motor para los 4 jobs, en streaming y en batch, integrado con Delta.

Costo: la latencia es la espera del micro-batch más su duración.

Kafka y no Kinesis

Spark open source no trae fuente de Kinesis; Kafka permite varios consumidores independientes con 7 días de retención.

Clave: la cuenta ordenante, para conservar el orden de cada cuenta.

58 / 58

variables idénticas al notebook, bit a bit

20.000 / 20.000

decisiones iguales al notebook

264 / 264

alertas del test completo (1.038.880 transacciones), con la misma matriz que el notebook: 141 de lavado real

El modelo de IA: un XGBoost que prioriza, no que bloquea

QUÉ EVALÚA

Probabilidad de lavado por transferencia, de 0 a 1



58 variables de comportamiento

Monto · Tiempo · Historia de la cuenta · Contraparte · Reenvío rápido

0,267
PR-AUC: unas 450 veces el azar

53 %
de las 264 alertas eran lavado real (141)

23 %
de los 617 casos de lavado detectados (141)

POR QUÉ ESTE

- Árboles con gradiente: estándar en fraude tabular; puntúan en milisegundos
- IBM AML: público y etiquetado; los datos de un banco son privados
- El reto del curso: servirlo idéntico (58 de 58 variables)

	Nuestro modelo	Industria (Stripe Radar, Visa)
Modelo	XGBoost, árboles con gradiente	Stripe Radar: red neuronal profunda
Datos	5 millones de transacciones sintéticas (IBM AML)	Miles de millones de pagos reales de toda la red
Variables	58	Más de 1.000 (Stripe)
Decide	Después del pago: avisa a la entidad y a los analistas	Durante la autorización y puede bloquear: < 100 ms (Stripe); score 0–99 (Visa)
Ciclo de vida	Entrenado una vez, umbral fijo	Reentrenamiento frecuente y reglas sobre el score

Exactly-once en el efecto, no en el transporte

Kafka entrega al menos una vez; si Spark falla, repite el micro-batch. Por eso cada efecto es idempotente.



5.000 / 5.000 transacciones en Bronze, Silver y risk_decisions, sin duplicados ni cuarentena

Decisión: estado en MongoDB (CP) y lakehouse en Delta Lake

MongoDB 7.0, replica set

Leer Silver en S3 toma segundos: no cabe en un micro-batch de 1 s

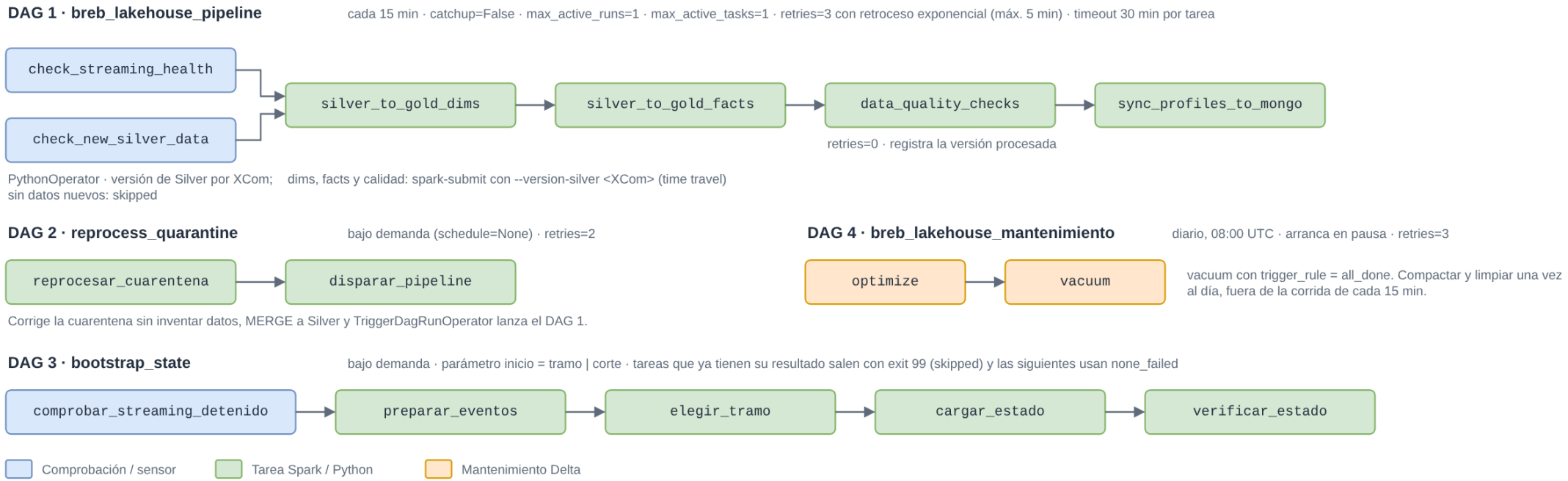
- Es CP: write y read concern majority
- Se sacrifica disponibilidad: si MongoDB no responde, el Job 2 espera en vez de decidir con datos viejos. En la EC2 hay un nodo; con 3 nodos, esa espera es la elección del primario, unos 2 s
- Cassandra descartada: sus contadores no son idempotentes

Delta Lake 3.3, en tres capas

Bronze	el 100 %, append-only, con la cuarentena marcada
Silver	sin duplicados y en UTC
Gold	estrella con SCD 2

- Log de commits en DynamoDB: varios procesos escriben la misma tabla
- Iceberg descartado: más complejo con streaming

Decisión: Airflow 3 para el batch; Trino y Athena para consultar



5 min 4 s
corrida completa del pipeline en AWS; OPTIMIZE y VACUUM van en un DAG diario aparte

10 / 10
casos de permisos por rol en Trino

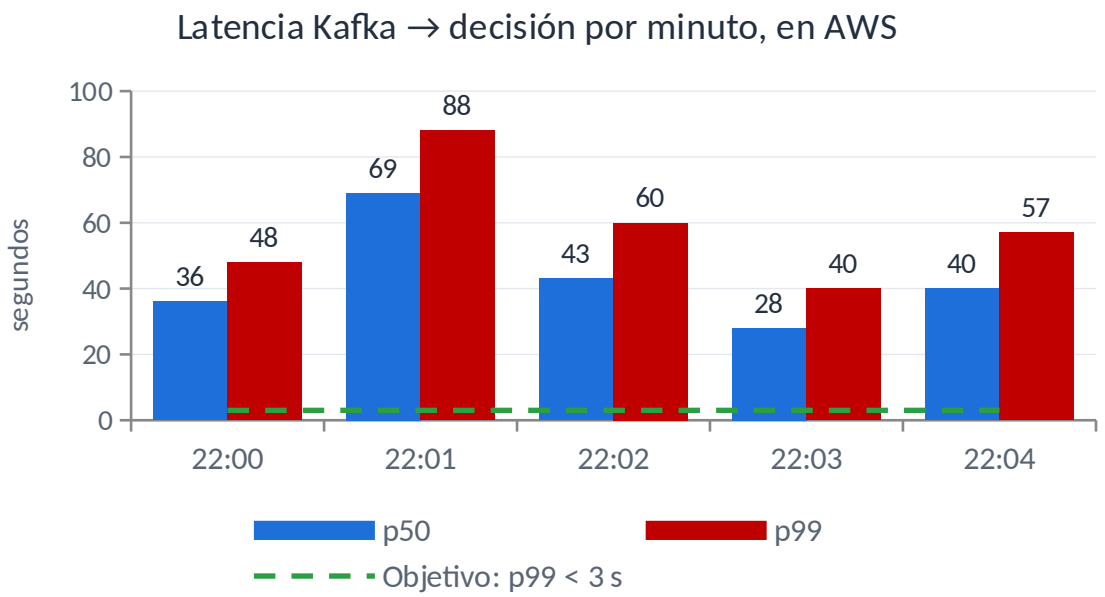
9 / 9
consultas del tablero, iguales en Athena

- El sensor fija la versión de Silver: un reintento no mezcla datos nuevos; la calidad corre antes de escribir en MongoDB.
- Analistas y Metabase solo ven Gold; cumplimiento lee todo sin modificar.

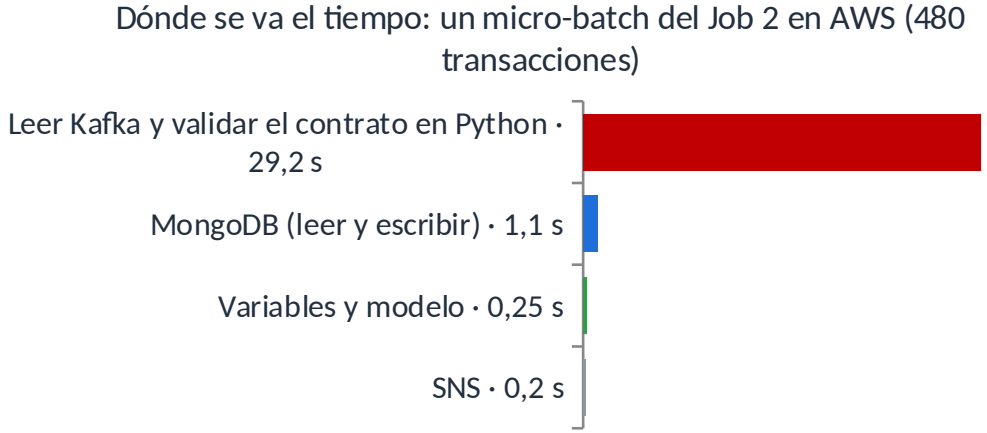
Resultados medidos en AWS (EC2 r5.large, 2 vCPU, 5.000 transacciones a 20 ev/s)

Requisito	Objetivo	Medido	Estado
RF1 · fuente → Kafka	p99 < 300 ms	p99 de 23 ms, 0 errores	✅ Cumple
RNF-01 · tasa del productor	1.000 ev/s	Probado a 20 ev/s en AWS, sin errores	🟡 Parcial
RNF-02 · Kafka → decisión	p99 < 3 s	p99 de 86 s (p50 de 40 s)	❌ No cumple
RF5 y RF6 · completitud	100 %, sin duplicados	5.000 de 5.000 en Bronze, Silver y risk_decisions	✅ Cumple
R8 · paridad con el notebook	Idéntica	Score en línea igual al de fuera de línea en 5.000 de 5.000; 264 alertas del test iguales al notebook	✅ Cumple
Alertas a la entidad	Por SNS y SQS	8 alertas; la entidad retiene los fondos y bloquea la cuenta	✅ Cumple
RF7 y RF9 · Gold con Airflow	Cada 15 min	Corrida en 5 min 4 s; precisión de 0,50 y recall de 0,40	✅ Cumple
RNF-06 · linaje	< 5 min	Linaje completo, de Kafka a la acción de la entidad, reconstruido en 19,5 s	✅ Cumple
RF11 · acceso por rol	Por rol	10 de 10 casos	✅ Cumple

Qué falló: la latencia de la decisión, y por qué



29 de 31 s se van en leer y validar el micro-batch



Otros errores encontrados y su arreglo

- AWS Academy no deja crear roles de IAM (iam:CreateRole denegado) → Usar LabRole con LabInstanceProfile; los contenedores toman las credenciales de la instancia
- Reiniciar con down -v sin perfiles dejaba los checkpoints viejos: los jobs no procesaban nada → Reiniciar con --profile '*'
- Limitar los offsets por micro-batch cambiaba las variables (226 de 9.204) → Sin tope; el Job 2 ordena cada micro-batch por seq

Qué cambiaríamos con más tiempo, y por qué

1

Validar el contrato con expresiones de Spark en vez de Python

Es el 95 % del micro-batch: 29 de 31 s en AWS

2

Pasar Kafka, Spark y Airflow a MSK, EMR y MWAA

Hoy comparten una EC2 de 2 vCPU; separados, cada uno escala por su cuenta

3

Probar la tasa nominal de 1.000 ev/s en AWS y declarar la tasa con 30 % de margen

En AWS solo se probó a 20 ev/s

4

Correr las pruebas de falla: bróker, ejecutor y primario de MongoDB

RNF-03, 04 y 05: diseñados, no medidos

5

Gold incremental con el Change Data Feed de Silver

Hoy el Job 4 compara contra todos los hechos

6

Secrets Manager, KMS con llave propia, autenticación en Trino y CI

En AWS ya hay rol de IAM, HTTPS y cifrado por defecto

Demo en AWS: qué vamos a mostrar (8 minutos)



0:00 — Consola de AWS

la EC2 con LabRole, los buckets de S3 y la tabla delta_log en DynamoDB

0:40 — Productor

publica en Kafka

1:00 — Streaming

Spark UI y el log del Job 2; los objetos nuevos en S3 y las alertas por SNS y SQS llegando a la entidad

3:45 — Airflow

arma Gold

6:15 — Gold

en Athena y en Metabase

7:15 — Linaje

de una alerta

Respaldo: qué pasa si algo falla

Solo si preguntan por fallas

Falla	Qué pasa
Cae un bróker	Hay un bróker en la EC2: los jobs se reinician y siguen desde el checkpoint, sin perder ni duplicar eventos.
Spark falla a mitad de un micro-batch	Repite con los mismos offsets; MongoDB parte del estado anterior y no se duplica nada.
Cae MongoDB	En la EC2 hay un solo nodo: el Job 2 espera a que se reinicie y sigue desde el checkpoint. Con 3 nodos se elige otro primario en unos 2 s, sin perder escrituras confirmadas.
SNS no responde	El micro-batch se repite, y la entidad descarta los duplicados.
Falla una tarea de Airflow	Hasta 3 reintentos sobre la misma versión de Silver.
Llegan eventos inválidos	Quedan en Bronze con is_valid = false, y reprocess_quarantine los corrige.